Arrays and ArrayLists

An array is a data structure used to store a collection of data.

Why do we need arrays?

Arrays are used when there is a need to use many variables of the same type. It can be defined as a sequence of objects of the same datatype.

Syntax: `datatype [] variable_name = new datatype [size];`

For eg: `int[] rollnos = new int [5]` — store 5 roll numbers

OR

`int [] rollnos = {51, 82, 13, 15, 16}`

Represent the type of data stored in the array

All the data should have the same datatype

INTERNAL WORKING OF AN ARRAY

- `int [] rollnos;` // declaration of array
  - ↳ Roll numbers are getting defined in stack
- `rollnos = new int [5];` // Initialization
  - ↳ Actual memory allocation happens here. Here, object is being created in heap memory.

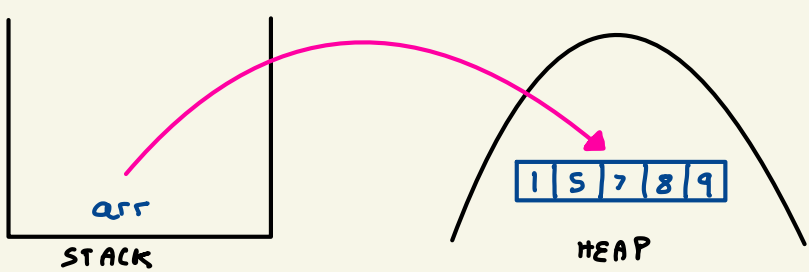
declaration of array — compile time

Initialization — run time

```
int [] arr = new int [5];
```

datatype — reference variable — creating object in heap memory

→ This concept is known as Dynamic memory allocation which means that memory is allocated at runtime.



INTERNAL REPRESENTATION OF ARRAY

- Internally in JAVA, memory allocation totally depends on JVM whether it can be continuous or not!

REASON 1: Objects are stored in heap memory.

REASON 2: In JLS (JAVA language specification), it is mentioned that heap objects are not continuous.

REASON 3: Dynamic memory allocation. Hence, array objects in JAVA may or may not be continuous (depends on JVM).

ARRAY INDEXING

Index →	0	1	2	3	4	5
arr →	3	8	9	10	53	33

arr[0] = 3 arr[2] = 9 arr[4] = 53

arr[1] = 8 arr[3] = 10 arr[5] = 33

To change a value in the array;

arr[4] = 99

New array will be:

3	8	9	10	99	33

NEW KEYWORD

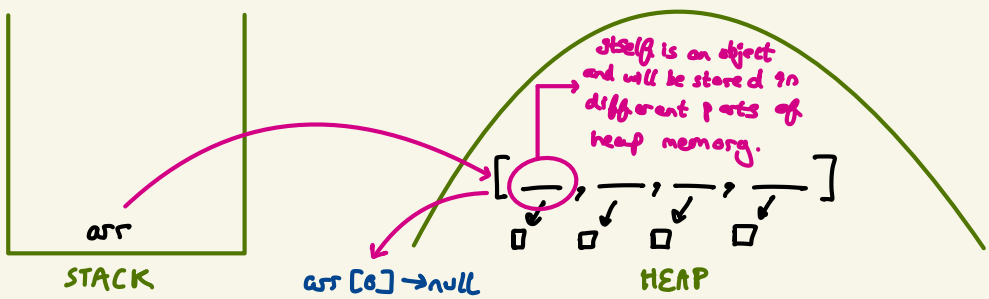
```
int[] arr = new int[5];
```

→ used to create an object

It will create an object in heap memory of array size 5.

* If we don't provide values in the array, it by default stores [0,0,0,0,0].

```
String[] arr = new String[4];
```



* primitives (int, char etc.) are stored in Stack.
* All other objects are stored in heap memory.
* In an array, since we can change the objects, hence they are mutable.
* Strings are immutable.

2D ARRAY

$$\begin{array}{c} 3 \\ \\ \end{array}\underbrace{\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}}_{3}$$

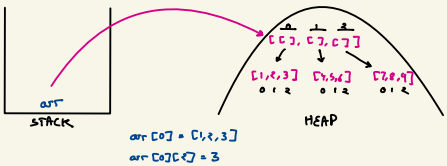
→ int [][] arr = new int [size][]

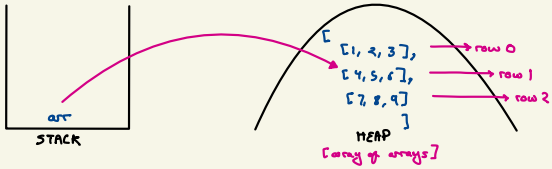
row — column

mandatory — not mandatory

OR

int [][] arr = {
{1, 2, 3},
{4, 5, 6},
{7, 8, 9}
}





OPERATIONS ON ARRAYS

1. Arrays.toString(Array) → Internally uses for loop and gives the output in proper format.
2. ArrayUtils.reverse(Array) → Used to reverse an array.
3. ArrayUtils.addAll(Array1, Array2) → Used to concatenate two arrays.
4. Arrays.asList(Array).contains("a") → Checks if an array contains a certain value. Returns a Boolean value.
5. Arrays.equals(Array1, Array2) → Method to check whether two array objects contain the same values. Returns bool.
6. Array.length → Calculates the length of an array.

ARRAYLISTS

Arraylist is a part of collection framework and is present in java.util.package. It provides us with dynamic arrays in Java. It is slower than standard arrays.

```
ArrayList <Integer> list = new ArrayList <>();
```

↳ add wrappers

- Size is fixed internally.
- Suppose arraylist gets filled by some amount.
  - → It will make an arraylist of say double the size of arraylist initially.
  - → Old elements are copied in the new arraylist.
  - → Old ones are deleted.

OPERATIONS

Method	Description	Return Type
charAt()	Returns the character at the specified index (position)	char
codePointAt()	Returns the Unicode of the character at the specified index	int
codePointBefore()	Returns the Unicode of the character before the specified index	int
codePointCount()	Returns the number of Unicode values found in a string.	int
compareTo()	Compares two strings lexicographically	int
compareToIgnoreCase()	Compares two strings lexicographically, ignoring case differences	int
concat()	Appends a string to the end of another string	String
contains()	Checks whether a string contains a sequence of characters	boolean
contentEquals()	Checks whether a string contains the exact same sequence of characters of the specified CharSequence or StringBuffer	boolean
copyValueOf()	Returns a String that represents the characters of the character array	String
endsWith()	Checks whether a string ends with the specified character(s)	boolean
equals()	Compares two strings. Returns true if the strings are equal, and false if not	boolean
equalsIgnoreCase()	Compares two strings, ignoring case considerations	boolean
format()	Returns a formatted string using the specified locale, format string, and arguments	String
getBytes()	Encodes this String into a sequence of bytes using the named charset, storing the result into a new byte array	byte[]
getChars()	Copies characters from a string to an array of chars	void
hashCode()	Returns the hash code of a string	int
indexOf()	Returns the position of the first found occurrence of specified characters in a string	int
intern()	Returns the canonical representation for the string object	String
isEmpty()	Checks whether a string is empty or not	boolean
lastIndexOf()	Returns the position of the last found occurrence of specified characters in a string	int
length()	Returns the length of a specified string	int
matches()	Searches a string for a match against a regular expression, and returns the matches	boolean
offsetByCodePoints()	Returns the index within this String that is offset from the given index by codePointOffset code points	int
regionMatches()	Tests if two string regions are equal	boolean
replace()	Searches a string for a specified value, and returns a new string where the specified values are replaced	String
replaceFirst()	Replaces the first occurrence of a substring that matches the given regular expression with the given replacement	String
replaceAll()	Replaces each substring of this string that matches the given regular expression with the given replacement	String
split()	Splits a string into an array of substrings	String[]
startsWith()	Checks whether a string starts with specified characters	boolean
subSequence()	Returns a new character sequence that is a subsequence of this sequence	CharSequence
substring()	Returns a new string which is the substring of a specified string	String
toCharArray()	Converts this string to a new character array	char[]
toLowerCase()	Converts a string to lower case letters	String
toString()	Returns the value of a String object	String
toUpperCase()	Converts a string to upper case letters	String
trim()	Removes whitespace from both ends of a string	String
valueOf()	Returns the string representation of the specified value	String